

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №2
по дисциплине
«Современные системы программирования»

Выполнила:

ст. гр. 320604
К. А. Хаджинова

Проверила:

Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ	3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ	4
3 ВЫПОЛНЕНИЕ РАБОТЫ	5
3.1 Подготовка среды и режим запуска	5
3.2 Статическая и экземплярная ссылки	5
3.3 Задание 1. Базовый пример с утечкой памяти	6
3.4 Задания 2 и 3. Класс мониторинга памяти и его тестирование	8
3.5 Анализ с помощью снимков памяти	10
3.6 Задание 4. Статические ссылки, утечки и поколения объектов	12
3.7 Задание 5. Исследование <i>Large Object Heap</i>	14
3.8 Задание 6. Устранение упаковки	15
3.9 Задание 7. Поиск утечек в коде	17
4 КОНТРОЛЬНЫЕ ВОПРОСЫ	20
4.1 Вопросы к заданию 4	20
4.2 Вопросы к заданию 5	20
4.3 Вопрос 1. Поколения	20
4.4 Вопрос 2. Статические ссылки	20
4.5 Вопрос 3. <i>LOH</i> и фрагментация	21
4.6 Вопрос 4. Событийные утечки	21
4.7 Вопрос 5. <i>Boxing</i> и производительность	21
ЗАКЛЮЧЕНИЕ	22

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы сборки мусора в *.NET*, работу с поколениями памяти, выявление утечек памяти с помощью профилировщика памяти, анализ *Large Object Heap (LOH)* и оптимизацию через устранение упаковки (*boxing elimination*).

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Сборщик мусора (*Garbage Collection*, *GC*) в *.NET* автоматически освобождает память объектов, на которые не осталось ссылок. Объекты распределены по поколениям: *Gen0* содержит молодые объекты, *Gen1* – промежуточные, *Gen2* – долгоживущие. Объекты размером более 85 000 байт размещаются в *Large Object Heap* (*LOH*). Сборку можно вызвать принудительно командами *GC.Collect()* и *GC.WaitForPendingFinalizers()*.

Типичные причины утечек памяти: неосвобожденные события и делегаты, статические ссылки, неправильная работа с *IDisposable*, зацикленные ссылки. Статическая ссылка общая для всех объектов класса и живет весь цикл работы приложения, поэтому объекты, достижимые из нее, сборщик не удаляет.

Упаковка (*boxing*) – сохранение значения простого типа как объекта в управляемой куче, распаковка (*unboxing*) – обратное извлечение значения. *GC* не вызывается при завершении программы, финализаторы не гарантируются, поэтому для неуправляемых ресурсов применяют *using* или явный вызов *Dispose()*.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Подготовка среды и режим запуска

Приложение *Lab2* создано как консольный проект *.NET* с целевой платформой *net10.0* (*SDK 10.0.401*). Все замеры выполнены в режиме *Release*, так как в режиме отладки время жизни локальных переменных искусственно продлевается и результаты работы сборщика искажаются. Запуск: *dotnet build -c Release*, затем *dotnet bin\Release\net10.0\Lab2.dll*.

Профилировщики *dotMemory* и *Visual Studio Diagnostic Tools* на компьютере не установлены, поэтому снимки памяти (*snapshot*) получены утилитой *dotnet-gcdump*, которая сохраняет состояние управляемой кучи процесса и формирует отчет по типам объектов.

3.2 Статическая и экземплярная ссылки

Методические указания содержат демонстрацию различия статической и экземплярной ссылок. Класс *StaticReferenceDemo* хранит общий статический список и собственный список в каждом экземпляре, класс *StaticFieldDemo* – общий и индивидуальный счетчики. Результат показан на рисунке 3.1.

```
> dotnet Lab2.dll
=== ОДНА статическая ссылка на ВСЕ объекты ===

1. Добавляем данные через разные объекты:

Объект 1:
    Статический список (общий): [100, 200, 300]
    Экземплярный список (свой): [1]

Объект 2:
    Статический список (общий): [100, 200, 300]
    Экземплярный список (свой): [2]

Объект 3:
    Статический список (общий): [100, 200, 300]
    Экземплярный список (свой): [3]

Статический список ОДИН для всех!
Экземплярные списки РАЗНЫЕ для каждого!

2. Демонстрация через статическое поле:

После операций:
Объект А:
    Статический счетчик (общий): 3
    Экземплярный счетчик (свой): 2
Объект В:
    Статический счетчик (общий): 3
    Экземплярный счетчик (свой): 1

Статический счетчик ОДИН для обоих объектов!
Экземплярные счетчики РАЗНЫЕ для каждого!
```

Рисунок 3.1 – Статическая и экземплярная ссылки

Статический список у трех объектов содержит одни и те же значения 100, 200 и 300, а экземплярные списки у каждого объекта свои. Статический счетчик после трех вызовов равен 3 для обоих объектов, индивидуальные счетчики равны 2 и 1.

3.3 Задание 1. Базовый пример с утечкой памяти

Класс *MemoryLeakExample* добавляет в статический список 1000 массивов по 10 КБ, класс *EventLeakExample* подписывает обработчики на событие без отписки. Метод *ClearMemoryLeak()* очищает список и вызывает сборку. Фрагмент кода приведен в листинге, полный текст программы вместе с вызовами методов находится в файле *Task1.cs*.

Листинг 3.1 – Классы утечек из задания 1

```
public class MemoryLeakExample
{
    private static List<byte[]> _staticList = new List<byte[]>();

    public static void CreateMemoryLeak()
    {
```

```

    // Создаем объекты, которые никогда не будут собраны
    for (int i = 0; i < 1000; i++)
    {
        _staticList.Add(new byte[1024 * 10]); // 10KB каждый
    }
}

public static void ClearMemoryLeak()
{
    _staticList.Clear();
    GC.Collect();
    GC.WaitForPendingFinalizers();
}
}

```

Программа последовательно выделяет память, создает утечки, сравнивает упаковку с обычным списком, выполняет очистку и показывает поколение объекта. Результат работы показан на рисунках 3.2 и 3.3.

```

> dotnet Lab2.dll
=== Лабораторная работа: Управление памятью ===

1. Выделение памяти:

=== После выделения ===
GC Generation: 1
Total Memory: 18 MB
Collection Count Gen0: 1
Collection Count Gen1: 1
Collection Count Gen2: 1
Working Set: 24 MB

2. Тест утечек:

=== После утечек ===
GC Generation: 2
Total Memory: 20 MB
Collection Count Gen0: 4
Collection Count Gen1: 4

```

Рисунок 3.2 – Результат выполнения задания 1 (начало)

После создания массивов объем управляемой памяти составил 18 МБ, после создания утечек 20 МБ, а число сборок поколений выросло, так как GC запускался при выделении памяти.

Продолжение вывода программы показано на рисунке 3.3.

```

Collection Count Gen2: 2
Working Set: 32 MB

3. Тест упаковки:
Время с упаковкой: 51ms
Время без упаковки: 7ms

4. После очистки:

=== Очищено ===
GC Generation: 2
Total Memory: 4 MB
Collection Count Gen0: 9
Collection Count Gen1: 7
Collection Count Gen2: 4
Working Set: 43 MB

5. Поколения объектов:
Начальное поколение: 0
После GC: 1

```

Рисунок 3.3 – Результат выполнения задания 1 (продолжение)

Добавление миллиона чисел в список объектов с упаковкой заняло 51 мс, в список целых чисел без упаковки – 7 мс. После очистки управляемая память уменьшилась до 4 МБ. Новый объект относится к поколению 0, а после вызова *GC.Collect()* переходит в поколение 1.

3.4 Задания 2 и 3. Класс мониторинга памяти и его тестирование

Класс *MemoryMonitor* реализует интерфейс *IDisposable*. Метод *AllocateMemory* выделяет блок памяти в мегабайтах и хранит его в списке, *AllocateLOHObjects* создает массивы больше 85 000 байт, *SimulateBoxing* добавляет в список объектов миллион целых чисел, *PrintMemoryInfo* выводит поколение, объем памяти и число сборок, *Cleanup* очищает список и вызывает сборку мусора, *Dispose* вызывает *Cleanup* один раз. Код класса приведен в листинге.

Листинг 3.2 – Класс *MemoryMonitor*

```

public class MemoryMonitor : IDisposable
{
    private bool _disposed;
    private List<byte[]> _allocatedMemory = new List<byte[]>();
    private Random _random = new Random();

    public void AllocateMemory(int sizeInMB)
    {
        // Выделяем память с возможной утечкой
        var data = new byte[sizeInMB * 1024 * 1024];
        _allocatedMemory.Add(data);
    }
}

```

```

public void AllocateLOHObjects(int count)
{
    for (int i = 0; i < count; i++)
    {
        // Объекты > 85KB
        var largeObject = new byte[90000 + i];
    }
}

public void SimulateBoxing(int count = 1000000)
{
    var list = new List<object>();
    for (int i = 0; i < count; i++)
    {
        list.Add(i); // Упаковка
    }
}

public void PrintMemoryInfo(string message = "")
{
    if (!string.IsNullOrEmpty(message))
        Console.WriteLine($"n=== {message} ===");

    Console.WriteLine($"GC Generation: {GC.GetGeneration(this)}");
    Console.WriteLine($"Total Memory: {GC.GetTotalMemory(false) / (1024 * 1024)} MB");
    Console.WriteLine($"Collection Count Gen0: {GC.CollectionCount(0)}");
    Console.WriteLine($"Collection Count Gen1: {GC.CollectionCount(1)}");
    Console.WriteLine($"Collection Count Gen2: {GC.CollectionCount(2)}");

    using (Process process = Process.GetCurrentProcess())
    {
        Console.WriteLine($"Working Set: {process.WorkingSet64 / (1024 * 1024)} MB");
    }
}

public void Cleanup()
{
    _allocatedMemory.Clear();
    GC.Collect();
    GC.WaitForPendingFinalizers();
}

public void Dispose()
{
    if (!_disposed)
    {
        Cleanup();
        _disposed = true;
    }
}
}

```

Тестовая программа (задание 3) создает объект в операторе *using*, выделяет 10 МБ, создает 100 объектов в *LOH*, измеряет время упаковки, принудительно собирает мусор и сравнивает с версией без упаковки. Результат показан на рисунке 3.4.

```
> dotnet Lab2.dll
=== Лабораторная работа: Управление памятью ===

1. Базовое выделение памяти:
GC Generation: 0
Total Memory: 14 MB
Collection Count Gen0: 10
Collection Count Gen1: 8
Collection Count Gen2: 5
Working Set: 43 MB

2. Создание объектов в LOH:
GC Generation: 0
Total Memory: 12 MB
Collection Count Gen0: 11
Collection Count Gen1: 9
Collection Count Gen2: 6
Working Set: 46 MB

3. Тест упаковки:
Время выполнения с упаковкой: 81ms

4. После принудительной сборки:
GC Generation: 2
Total Memory: 0 MB
Collection Count Gen0: 17
Collection Count Gen1: 13
Collection Count Gen2: 8
Working Set: 35 MB

5. Оптимизированная версия (без упаковки):
Время выполнения без упаковки: 6ms
```

Рисунок 3.4 – Результат тестирования класса *MemoryMonitor*

После выделения 10 МБ объем управляемой памяти составил 14 МБ, после создания объектов *LOH* – 12 МБ, так как часть объектов без ссылок уже собрал сборщик. После принудительной сборки показание упало до 0 МБ. Упаковка заняла 81 мс, версия без упаковки – 6 мс.

3.5 Анализ с помощью снимков памяти

Для анализа предусмотрен режим *snapshots*, который останавливается на трех этапах: начальное состояние, после создания утечек и объектов *LOH* (статический список из 1000 массивов по 10 КБ, 100 подписок по 100 КБ и 100 массивов *LOH* по 100 КБ), после отписки и очистки списка. На каждой паузе командой *dotnet-gcdump collect* снимается снимок кучи. Отчеты по снимкам показаны на рисунках 3.5, 3.6 и 3.7.

```
> dotnet-gcdump report snapshot1.gcdump (снимок 1: начальное состояние)
326,352 GC Heap bytes
4,026 GC Heap objects

Object Bytes      Count Type
31,470      1 System.String (Bytes > 10K) [System.Private.CoreLib.dll]
22,088      1 Entry<System.Int32,System.Diagnostics.Tracing.EventSource+EventMetadata>[
] (Bytes > 10K) [System.Private.CoreLib.dll]
8,218       1 System.Char[] (Bytes > 1K) [System.Private.CoreLib.dll]
8,184       1 System.Object[] (Bytes > 1K) [System.Private.CoreLib.dll]
6,216       1 Entry<System.Runtime.CompilerServices.VirtualDispatchHelpers+VirtualResol
utionData,System.IntPtr>[] (Bytes > 1K) [System.Private.CoreLib.dll]
4,120       1 System.Byte[] (Bytes > 1K) [System.Private.CoreLib.dll]
3,120       1 System.Int32[] (Bytes > 1K) [System.Private.CoreLib.dll]
2,424       1 System.String[] (Bytes > 1K) [System.Private.CoreLib.dll]
2,064       1 System.Reflection.RuntimeMethodInfo[] (Bytes > 1K) [System.Private.CoreL
ib.dll]
448         2 System.Globalization.CultureInfo [System.Private.CoreLib.dll]
400         1 System.Diagnostics.Tracing.RuntimeEventSource [System.Private.CoreLib.dl
l]
288         1 Entry<System.String,System.Object>[] [System.Private.CoreLib.dll]
```

Рисунок 3.5 – Снимок 1: начальное состояние

В начальном состоянии управляемая куча занимает 326 352 байта и содержит 4026 объектов; крупных массивов *Byte[]* нет.

Состояние после создания утечек и объектов *LOH* показано на рисунке 3.6.

```
> dotnet-gcdump report snapshot2.gcdump (снимок 2: после утечек и объектов LOH)
31,073,630 GC Heap bytes
5,077 GC Heap objects

Object Bytes      Count Type
102,424     200 System.Byte[] (Bytes > 100K) [System.Private.CoreLib.dll]
31,470      1 System.String (Bytes > 10K) [System.Private.CoreLib.dll]
22,088      1 Entry<System.Int32,System.Diagnostics.Tracing.EventSource+EventMetadata>[
] (Bytes > 10K) [System.Private.CoreLib.dll]
10,264     1,000 System.Byte[] (Bytes > 10K) [System.Private.CoreLib.dll]
8,218       1 System.Char[] (Bytes > 1K) [System.Private.CoreLib.dll]
8,216       2 System.Byte[][] (Bytes > 1K) [System.Private.CoreLib.dll]
6,216       1 Entry<System.Runtime.CompilerServices.VirtualDispatchHelpers+VirtualResol
utionData,System.IntPtr>[] (Bytes > 1K) [System.Private.CoreLib.dll]
4,120       1 System.Byte[] (Bytes > 1K) [System.Private.CoreLib.dll]
3,120       1 System.Int32[] (Bytes > 1K) [System.Private.CoreLib.dll]
2,424       1 System.String[] (Bytes > 1K) [System.Private.CoreLib.dll]
2,064       1 System.Reflection.RuntimeMethodInfo[] (Bytes > 1K) [System.Private.CoreL
ib.dll]
1,048       2 System.Object[] (Bytes > 1K) [System.Private.CoreLib.dll]
```

Рисунок 3.6 – Снимок 2: после создания утечек и объектов *LOH*

Куча выросла до 31 073 630 байт и 5077 объектов. В отчете видны 200 массивов *Byte[]* размером около 100 КБ (100 захваченных обработчиками и 100 объектов *LOH*) и 1000 массивов по 10 КБ из статического списка.

Состояние после отписки и очистки списка показано на рисунке 3.7.

```

> dotnet-gcdump report snapshot3.gcdump (снимок 3: после отписки и очистки)
10,681,342 GC Heap bytes
4,676 GC Heap objects

Object Bytes      Count Type
102,424      1 System.Byte[] (Bytes > 100K) [System.Private.CoreLib.dll]
31,470       1 System.String (Bytes > 10K) [System.Private.CoreLib.dll]
22,088       1 Entry<System.Int32,System.Diagnostics.Tracing.EventSource+EventMetadata>[
] (Bytes > 10K) [System.Private.CoreLib.dll]
10,264      1,000 System.Byte[] (Bytes > 10K) [System.Private.CoreLib.dll]
8,218        1 System.Char[] (Bytes > 1K) [System.Private.CoreLib.dll]
8,216        2 System.Byte[][] (Bytes > 1K) [System.Private.CoreLib.dll]
8,184        1 System.Object[] (Bytes > 1K) [System.Private.CoreLib.dll]
6,216        1 Entry<System.Runtime.CompilerServices.VirtualDispatchHelpers+VirtualResol
utionData,System.IntPtr>[] (Bytes > 1K) [System.Private.CoreLib.dll]
4,120        1 System.Byte[] (Bytes > 1K) [System.Private.CoreLib.dll]
3,120        1 System.Int32[] (Bytes > 1K) [System.Private.CoreLib.dll]
2,424        1 System.String[] (Bytes > 1K) [System.Private.CoreLib.dll]
2,064        1 System.Reflection.RuntimeMethodInfo[] (Bytes > 1K) [System.Private.CoreL
ib.dll]

```

Рисунок 3.7 – Снимок 3: после отписки и очистки

Куча уменьшилась до 10 681 342 байт. Остался один массив около 100 КБ, а 1000 массивов по 10 КБ сохранились: на них ссылается статический список, который не очищался. Сравнение снимков подтверждает, что отписка от события и очистка списка освободили память, а статическая ссылка ее удерживает.

3.6 Задание 4. Статические ссылки, утечки и поколения объектов

Класс *StaticLeakDemo* хранит статический список *List<byte[]>*, метод *AddData()* добавляет 1000 объектов по 10 КБ, метод *CheckMemory()* выводит размер памяти, число объектов в списке и его поколение. Класс *EventLeakDemo* содержит событие, метод *Subscribe()* подписывает обработчик, удерживающий большой объект (100 КБ), метод *Unsubscribe()* снимает подписки. Код приведен в листинге.

Листинг 3.3 – Классы *StaticLeakDemo* и *EventLeakDemo*

```

public class StaticLeakDemo
{
    private static List<byte[]> _staticList = new List<byte[]>();

    // Добавляет 1000 объектов по 10KB в статический список
    public static void AddData()
    {
        for (int i = 0; i < 1000; i++)
            _staticList.Add(new byte[1024 * 10]);
    }

    public static void CheckMemory()
    {

```

```

        Console.WriteLine($" Текущий размер памяти: {GC.GetTotalMemory(false) / 1024}
KB");
        Console.WriteLine($" Количество объектов в статическом списке:
{_staticList.Count}");
        Console.WriteLine($" Поколение статического списка:
{GC.GetGeneration(_staticList)}");
    }
}

// Событийная утечка
public class EventLeakDemo
{
    public event EventHandler? DemoEvent;
    private List<EventHandler> _handlers = new List<EventHandler>();

    // Подписка на событие; большой объект создаётся в обработчике и живёт, пока есть
подписка
    public void Subscribe()
    {
        var bigObject = new byte[1024 * 100];
        EventHandler handler = (sender, e) => { bigObject[0]++; };
        DemoEvent += handler;
        _handlers.Add(handler);
    }

    // Отписка от всех подписок
    public void Unsubscribe()
    {
        foreach (var handler in _handlers)
            DemoEvent -= handler;
        _handlers.Clear();
    }
}

```

Дополнительно исследуется перемещение объекта между поколениями и размещение массивов разного размера. Результат показан на рисунке 3.8.

```

> dotnet Lab2.dll
=== Задание 4: статическая утечка ===
До добавления данных:
    Текущий размер памяти: 307 КВ
    Количество объектов в статическом списке: 0
    Поколение статического списка: 0
После AddData():
    Текущий размер памяти: 10346 КВ
    Количество объектов в статическом списке: 1000
    Поколение статического списка: 1
После GC.Collect() (объекты не удаляются):
    Текущий размер памяти: 10322 КВ
    Количество объектов в статическом списке: 1000
    Поколение статического списка: 2

=== Задание 4: событийная утечка ===
До подписок: 10330 КВ
После 100 подписок: 20357 КВ
После GC.Collect() (подписки живы): 20347 КВ
После отписки и GC.Collect(): 10339 КВ

=== Задание 4: сравнение поколений объектов ===
Новый объект: Gen0
После GC.Collect() №1: Gen1
После GC.Collect() №2: Gen2
После GC.Collect() №3: Gen2
Массив 1000 байт: Gen0
Массив 50 000 байт: Gen0
Массив 100 000 байт (> 85KB): Gen2

```

Рисунок 3.8 – Результат выполнения задания 4

После *AddData()* память выросла с 307 КБ до 10 346 КБ, а после *GC.Collect()* осталась на уровне 10 322 КБ, так как статический список удерживает все объекты. После 100 подписок память выросла с 10 330 КБ до 20 357 КБ и не уменьшилась после сборки, но после отписки вернулась к 10 339 КБ. Новый объект последовательно перешел из *Gen0* в *Gen1* и *Gen2*, массив 100 000 байт сразу оказался в *Gen2*.

3.7 Задание 5. Исследование *Large Object Heap*

Метод выделяет 100 массивов по 100 КБ и измеряет память до выделения, после выделения и после сборки. Затем создаются массивы 80 000 и 90 000 байт и объекты размером 10, 50, 90 и 200 КБ, для каждого определяется поколение. Результат показан на рисунке 3.9.

```

> dotnet Lab2.dll
=== Задание 5: объекты в LOH ===
Память до выделения: 10490 KB
Память после выделения 100 x 100KB: 20356 KB
Память после GC.Collect() (ссылки удерживаются): 20341 KB
Память после очистки списка и GC.Collect(2): 10439 KB

Сравнение размещения:
small (80 000 байт): Gen0
large (90 000 байт): Gen2

=== Задание 5: мониторинг LOH ===
Объект 10KB: Gen0
Объект 50KB: Gen0
Объект 90KB: Gen2
Объект 200KB: Gen2

```

Рисунок 3.9 – Результат выполнения задания 5

Память после выделения выросла с 10 490 КБ до 20 356 КБ и не изменилась при сборке, пока ссылки удерживались. После очистки списка и *GC.Collect(2)* она вернулась к 10 439 КБ. Массивы 80 000, 10 и 50 КБ находятся в *Gen0*, а массивы 90 000 байт, 90 и 200 КБ сразу в *Gen2*, так как попадают в *LOH*.

3.8 Задание 6. Устранение упаковки

Методы *ProcessWithBoxing()* и *ProcessWithoutBoxing()* добавляют миллион чисел в *List<object>* и *List<int>*. Время измеряется *Stopwatch*, память – разницей *GC.GetTotalMemory()* до и после операции. Затем код с *ArrayList* (класс *UnoptimizedCode*) переписан на *List<int>* (класс *OptimizedCode*). Код классов приведен в листинге.

Листинг 3.4 – Классы *UnoptimizedCode* и *OptimizedCode*

```

public class UnoptimizedCode
{
    private ArrayList _data = new ArrayList();

    public void AddData(int value)
    {
        _data.Add(value); // Упаковка!
    }

    public int GetSum()
    {
        int sum = 0;
        foreach (object item in _data) // Распаковка!
        {
            sum += (int)item;
        }
        return sum;
    }
}

```

```

    }
}

public class OptimizedCode
{
    private List<int> _data = new List<int>();

    public void AddData(int value)
    {
        _data.Add(value); // Без упаковки
    }

    public int GetSum()
    {
        int sum = 0;
        foreach (int item in _data)
        {
            sum += item;
        }
        return sum;
    }
}

```

Результаты сравнения показаны на рисунке 3.10.

```

> dotnet Lab2.dll
=== Задание 6.1: сравнение производительности (1 000 000 операций) ===
Метод                Время, мс    Память, КБ
ProcessWithBoxing      84          31618
ProcessWithoutBoxing   4           6296

=== Задание 6.2: оптимизация кода с ArrayList ===
Класс                Время, мс    Память, КБ
UnoptimizedCode       218         31635
OptimizedCode          14          6292

```

Рисунок 3.10 – Результат выполнения задания 6

Вариант с упаковкой выполнялся за 84 мс и занял 31 618 КБ, вариант без упаковки – за 4 мс и 6 296 КБ. Для кода с *ArrayList* получено 218 мс и 31 635 КБ, для оптимизированного кода – 14 мс и 6 292 КБ.

Те же результаты сведены в таблице 3.1.

Таблица 3.1 – Сравнение операций с упаковкой и без нее (1 000 000 операций)

Вариант	Время, мс	Память, КБ
<i>ProcessWithBoxing</i> (<i>List<object></i>)	84	31 618
<i>ProcessWithoutBoxing</i> (<i>List<int></i>)	4	6 296
<i>UnoptimizedCode</i> (<i>ArrayList</i>)	218	31 635

Вариант	Время, мс	Память, КБ
<i>OptimizedCode (List<int>)</i>	14	6 292

Ускорение объясняется тем, что при упаковке каждое число превращается в отдельный объект в куче: создается миллион объектов, растет нагрузка на сборщик мусора, а чтение требует распаковки. Список *List<int>* хранит числа в одном непрерывном массиве без дополнительных объектов, поэтому в данном замере он быстрее примерно в 15–20 раз.

3.9 Задание 7. Поиск утечек в коде

В классе *BuggyCode* найдены утечки. Каждая причина и исправление приведены в таблице 3.2. Исправленная версия реализует *IDisposable*: в методе *Dispose* выполняется отписка от события, удаление потока из статического кэша и его освобождение. Код классов приведен в листинге.

Таблица 3.2 – Утечки в классе *BuggyCode*

Место	Причина	Исправление
<i>_globalEvent += OnEvent</i>	Статическое событие хранит ссылку на каждый созданный объект, подписка не снимается	Отписка <i>_globalEvent -= OnEvent</i> в методе <i>Dispose</i>
<i>_cache.Add(new MemoryStream(1024))</i>	Статический список хранит потоки и не освобождает их	Удаление потока из кэша и вызов <i>Dispose</i> у потока
<i>Byte[] _data</i> (1 МБ)	Массив достижим через подписку на событие и поэтому не собирается	Освобождается после отписки от события

Листинг 3.5 – Классы *BuggyCode* и *FixedCode*

```
public class BuggyCode
{
    private static List<IDisposable> _cache = new List<IDisposable>();
    private static event EventHandler? _globalEvent;
    private byte[] _data = new byte[1024 * 1024];

    public BuggyCode()
    {
        _globalEvent += OnEvent;           // Утечка 1: подписка на статическое событие без отписки
        _cache.Add(new MemoryStream(1024)); // Утечка 2: статический кэш хранит потоки и не освобождает их
    }

    private void OnEvent(object? sender, EventArgs e)
    {
        var temp = new byte[1024 * 100];
    }
}
```

```

    }

    public void SubscribeAll()
    {
        for (int i = 0; i < 1000; i++)
        {
            new BuggyCode();           // Умечка 3: _data (1MB) каждого объекта
            // удерживается через событие
        }
    }

    public static int CacheCount => _cache.Count;
}

public class FixedCode : IDisposable
{
    private static List<IDisposable> _cache = new List<IDisposable>();
    private static event EventHandler? _globalEvent;
    private byte[]? _data = new byte[1024 * 1024];
    private MemoryStream _stream = new MemoryStream(1024);

    public FixedCode()
    {
        _globalEvent += OnEvent;
        _cache.Add(_stream);
    }

    private void OnEvent(object? sender, EventArgs e)
    {
        var temp = new byte[1024 * 100];
    }

    public void SubscribeAll()
    {
        for (int i = 0; i < 1000; i++)
        {
            using (new FixedCode())    // объект освобождается сразу после использования
            {
            }
        }
    }

    public void Dispose()
    {
        _globalEvent -= OnEvent;       // отписка от события
        _cache.Remove(_stream);        // удаление из статического кэша
        _stream.Dispose();             // освобождение потока
        _data = null;
    }

    public static int CacheCount => _cache.Count;
}

```

Результат запуска показан на рисунке 3.11.

```
> dotnet Lab2.dll task7

=== Задание 7: код с утечками ===
Память до: 0 MB
Память после SubscribeAll() и GC.Collect(): 1002 MB
Объектов в статическом кэше: 1001

=== Задание 7: исправленная версия ===
Память до: 1002 MB
Память после SubscribeAll() и GC.Collect(): 1002 MB
Объектов в статическом кэше: 0
```

Рисунок 3.11 – Результат выполнения задания 7

После создания 1001 объекта *BuggyCode* и сборки мусора память составила 1002 МБ, в статическом кэше остался 1001 объект. Версия *FixedCode* после вызова *Dispose* не оставила объектов в кэше и не увеличила занятую память (показание 1002 МБ осталось тем же).

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Вопросы к заданию 4

Объекты в обработчике события не удаляются, потому что событие хранит ссылку на делегат подписчика, а делегат ссылается на все, что он захватил. Пока подписка не снята, сборщик считает эти объекты достижимыми. После отписки ссылка исчезает, объекты становятся недостижимыми, и следующая сборка освобождает занятую ими память, что подтверждено в задании 4.

4.2 Вопросы к заданию 5

Объекты больше 85 000 байт попадают в *LOH*, потому что их перемещение при уплотнении кучи было бы слишком дорогим. По этой же причине *LOH* обычно не уплотняется: освобожденные участки остаются «дырами». Фрагментация *LOH* приводит к тому, что большой массив нельзя разместить, хотя суммарно свободной памяти достаточно, растет потребление памяти и время сборок. Куча *LOH* собирается вместе с поколением 2, то есть при полной сборке.

4.3 Вопрос 1. Поколения

Объекты, пережившие сборку, переходят из *Gen0* в *Gen1* и затем в *Gen2*. Молодые объекты собираются часто и дешево, долгоживущие – редко. Если объект ошибочно живет долго, например лежит в статическом списке, он оказывается в *Gen2* и освобождается только при полной сборке, что и приводит к утечке памяти.

4.4 Вопрос 2. Статические ссылки

Статическая ссылка принадлежит классу, а не экземпляру, живет до завершения приложения и является корнем для сборщика мусора. Все, что достижимо из статической коллекции, считается используемым. Поэтому даже небольшие объекты, накапливаясь в статической коллекции, постепенно занимают большой объем памяти.

4.5 Вопрос 3. *LOH* и фрагментация

LOH не уплотняется из-за стоимости копирования больших блоков. Освобожденные блоки между живыми объектами образуют фрагменты. Если программа часто создает и удаляет массивы разного размера, свободная память дробится, новые большие массивы не помещаются, и сборщик вынужден увеличивать кучу.

4.6 Вопрос 4. Событийные утечки

Подписчик, не отписавшийся от события, остается в списке вызова источника события. Если источник живет долго (статический объект), подписчик и все, на что он ссылается, не освобождаются. Исправление – отписываться оператором `-=`, например в методе *Dispose*. Безопасный пример: `source.Changed += Handler`; при завершении работы `source.Changed -= Handler`;

4.7 Вопрос 5. *Boxing* и производительность

Упаковка – помещение значения простого типа в объект в куче. Она создает дополнительные объекты, нагружает сборщик и требует распаковки при чтении. Код с *ArrayList* или *List<object>* оптимизируют заменой на типизированный *List<int>*, как в задании 6. В выполненной работе версия без упаковки оказалась быстрее и использовала меньше памяти.

ЗАКЛЮЧЕНИЕ

Изучены механизмы сборки мусора *.NET*: поколения объектов, *LOH*, принудительная сборка. Показано, что статическая коллекция и неснятые подписки на события удерживают память и после вызова *GC.Collect()*, а после отписки память освобождается. Объекты размером более 85 000 байт сразу попадают в *Gen2 (LOH)*. Три снимка памяти, полученные утилитой *dotnet-gcdump*, подтвердили рост и последующее уменьшение управляемой кучи. Устранение упаковки сократило время добавления миллиона элементов с 84 до 4 мс. Найдены и исправлены утечки в классе *BuggyCode*. Цель работы достигнута.